

SQL to PySpark and Delta Lake

Use familiar SQL to understand Spark, then build the first governed Delta table.

Week 5 Day 2

SQL skills become distributed data engineering.

Your SQL knowledge already works here

SELECT → WHERE → GROUP BY → WINDOW

Databricks SQL

Write relational logic in the notation you know. Use the SQL Editor or a %sql notebook cell.

PySpark DataFrames

Express the same logical operations through Python and compose them programmatically.

One execution engine

Both plans can run through Spark and produce governed tables in Unity Catalog.

Today changes the execution context, not the meaning of clean relational logic.

Choose the smallest engine that fits the job

pandas

Best fit

Local, bounded data
Fast exploration
Rich Python ecosystem

Watch for

All rows live in one Python process

Databricks SQL

Best fit

Relational transformations
Analyst collaboration
Clear business logic

Watch for

Programmatic workflows may need Python

PySpark

Best fit

Distributed data
Reusable programmatic pipelines
Complex transformations

Watch for

Driver collection and unnecessary complexity

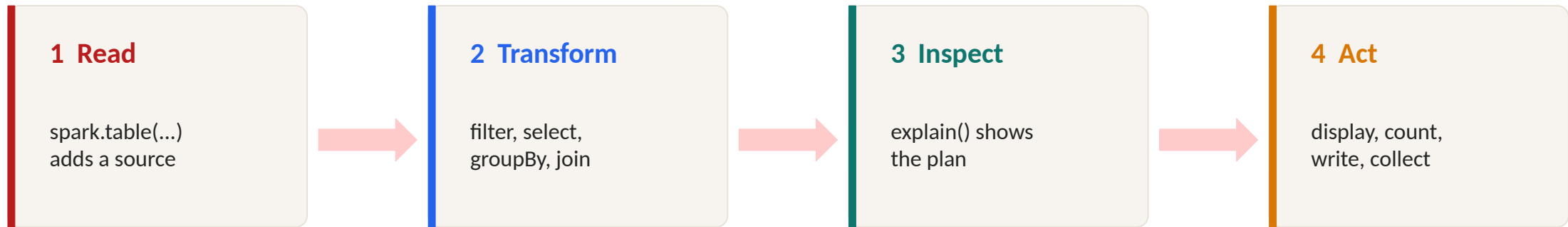
Scale is one factor. Clarity, maintainability, and the operating model matter too.

01

Spark plans before it works

Lazy evaluation separates describing a transformation from executing it.

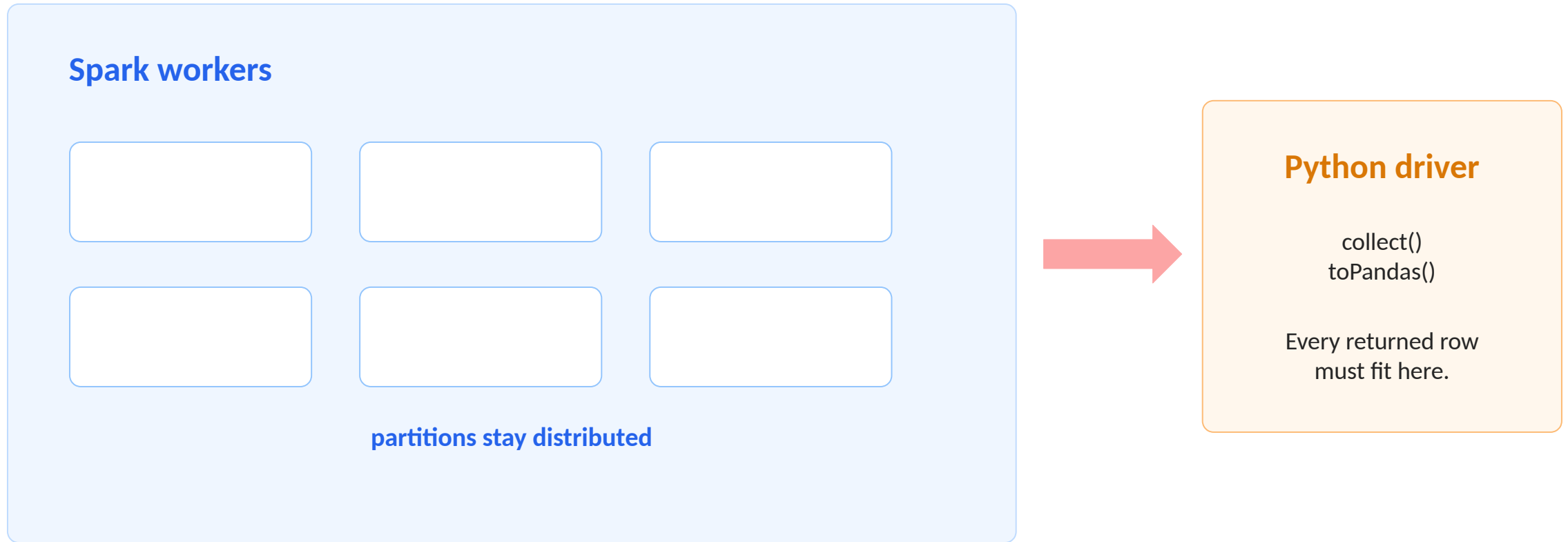
A DataFrame is a plan, not a box of rows



Only the action asks Spark to execute enough of the plan to produce a result.

Predict first: has the source been scanned yet?

Distributed work becomes risky when everything returns to Python



Safe pattern: filter and aggregate in Spark, then collect only a deliberately small result.

Magic commands change how a cell participates

%sql

Run SQL in this cell

%python

Run Python from another default language

%fs

Use file-system utility shorthand

%run

Execute another notebook inline

dbutils.widgets

Accept notebook or job parameters

spark.sql

Create a Spark DataFrame from SQL text

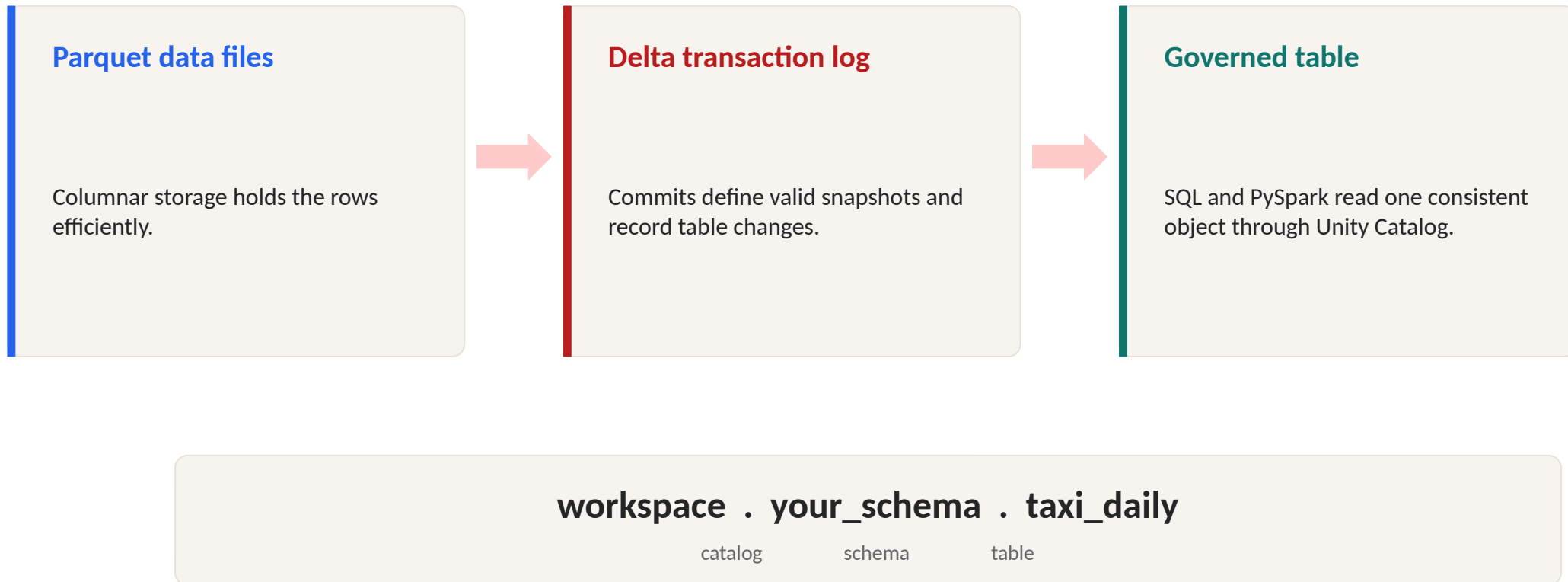
%run shares a notebook context. A Lakeflow Job task creates an observable runtime boundary.

02

Delta makes files behave like a table

The transaction log adds reliable snapshots, metadata, and change operations.

Delta adds a transaction layer around data files



Learn, imitate, build, then defend

1

Walkthrough

SQL to PySpark, lazy evaluation, widgets

2

Code-along

Read the plan and query profile

3

Independent build

Clean, aggregate, window, validate

4

Delta table

Create, inspect, update, time travel

5

Engine defense

Choose pandas, SQL, or Spark

A strong engineer can explain the boundary

Explain this pipeline in five sentences

- 1 Why did the DataFrame definition finish quickly?
- 2 Which line triggered execution?
- 3 Where could driver memory become a problem?
- 4 What did Delta add to the result?
- 5 Why was this small table not manually partitioned?